VLM versus Random Projection as Observation Encoder for Flow-Matching Robot Policies on CALVIN D→D

Experiment Report

Yasuyuki Matsushita

April 22, 2026

Contents

1 Introduction **2**

2 Methods **2**
2.1 FLOWER VLA (Florence-2-large + Flow DiT) 2
2.2 Random Projection VLA (RandomProj + Flow DiT) 3
2.3 Relationship between the Two Variants 3

3 Experimental Setup **3**
3.1 Hardware and Software 3
3.2 Dataset 4
3.3 Training Hyperparameters 4

4 Results **4**
4.1 Training and Validation Accuracy 4
4.1.1 Per-Epoch Validation Loss 5
4.1.2 Key Observations 5
4.2 CALVIN Long-Horizon Success Rate 5
4.3 Inference Latency 6
4.4 Memory Footprint 6

5 Discussion **6**
5.1 Contribution of the VLM Encoder 6
5.2 Why Does the VLM Matter? 6
5.3 Practical Implications 7

6 Conclusion **7**

1 Introduction

Vision-Language-Action (VLA) models combine large pre-trained Visual Language Models (VLMs) with learned action decoders to produce language-conditioned robot policies. FLOWER VLA [1] uses **Florence-2-large** [**?**] as its VLM backbone and a **Flow Matching Diffusion Transformer (DiT)** as the action model. While the combination achieves strong performance on the CALVIN benchmark [2], the role of the VLM is not isolated: does the VLM's *semantic* visual-language understanding matter, or is any fixed-dimensional encoding sufficient?

This report answers that question by training a **random projection baseline**: the Florence-2 encoder is replaced by a completely frozen random linear projection (image patches → 1024-D tokens; BERT tokens → 1024-D embeddings). The same DiT action model is then trained from scratch on the CALVIN D→D dataset.

We compare the two variants on:

- **Training / validation accuracy** (flow-matching MSE loss)
- **CALVIN long-horizon success rate** (pending: requires Linux)
- **Inference latency** (ms per policy step)
- **Memory footprint** (model weight size; peak VRAM)

2 Methods

2.1 FLOWER VLA (Florence-2-large + Flow DiT)

Architecture. FLOWER VLA consists of two main components:

1. **Visual-Language Encoder**: Florence-2-large (≈ 570 M encoder parameters after removing the decoder). Given a static RGB image, a wrist-camera image, and a text instruction, the model encodes them jointly via a shared Transformer encoder.
   - Images are processed through a DaViT vision tower, producing dense visual tokens ($49+49 = 98$ patch tokens per image pair).
   - The text instruction is tokenised with the Florence-2 tokeniser (BPE, vocab 51,289) and embedded.
   - Visual tokens, a learned `<Flow>` prompt token, and text tokens are concatenated and passed through the shared encoder.
2. **Flow Matching DiT**: 18 Transformer blocks (dim 1024, 16 heads) that predict a rectified-flow velocity field over 10-step action chunks. The DiT conditions on the VLM encoder output via cross-attention and AdaLN. It predicts Δend-effector 7-DoF actions.

Initialisation. Florence-2-large weights are loaded from HuggingFace (`microsoft/Florence-2-large`). The DiT is initialised from FLOWER pretraining (`mbreuss/flower_vla_pret`, step 360 k), giving the model a strong prior over robot manipulation dynamics.

Total parameters. ≈ 947 M (all trainable during fine-tuning).

2.2 Random Projection VLA (RandomProj + Flow DiT)

Architecture. The Florence-2 encoder is replaced by two **frozen random linear layers**:

1. **Random Patch Encoder**: Each 112×112 RGB image is divided into 7×7 = 49 non-overlapping 16×16 patches. Each patch vector (dimension $3 \times 16 \times 16 = 768$) is projected to 1024-D by a fixed random weight $W_{\text{img}} \in \mathbb{R}^{768 \times 1024}$ drawn from $\mathcal{N}(0,\, 1/768)$ and never updated. Two cameras yield $49 + 49 = 98$ visual tokens.

2. **Random Text Encoder**: The instruction is tokenised with `bert-base-uncased` (WordPiece, vocab 30 522). Each token index is looked up in a fixed random embedding table $E \in \mathbb{R}^{30{,}522 \times 1024}$, drawn from $\mathcal{N}(0,\, 0.02^2)$.

There is **no Transformer encoder**: random patch embeddings are concatenated directly with the (also random) text embeddings and a zero prompt token, then fed straight into the DiT.

DiT. **Identical** to FLOWER VLA (18 layers, dim 1024, 16 heads, RoPE, cross-attention), but **initialised randomly** (no pretraining).

Total parameters. ≈ 408 M total; 376 M trainable (DiT) + 32 M frozen (random encoders).

2.3 Relationship between the Two Variants

Table 1 summarises the architectural differences.

Component	FLOWER VLA	RandomProj VLA
Image encoder	DaViT + F2 encoder (trainable)	Frozen random proj. $\mathcal{N}(0, 1/768)$
Text encoder	BPE tokeniser + F2 encoder	WordPiece + frozen random embed. $\mathcal{N}(0, 0.02^2)$
Encoder transformer	Shared 12-layer Transformer	**None**
Token dim	1024	1024
DiT	18 layers, dim 1024, 16 heads	*identical*
DiT init	FLOWER pretraining (step 360k)	Random (scratch)
Encoder params	≈571 M (trainable)	32 M (frozen)
DiT params	≈376 M (trainable)	376 M (trainable)

Table 1: Architectural comparison.

3 Experimental Setup

3.1 Hardware and Software

Component	Details
OS	Windows 11 (training), Linux planned for eval
GPU	NVIDIA GeForce RTX 4090 (24 GB VRAM)
conda env	`flower_cal` (Python 3.10)
PyTorch	2.6.0+cu124
pytorch-lightning	2.0.8
CUDA	12.4

Table 2: Hardware and software environment.

3.2 Dataset

Both models are evaluated on the CALVIN D→D split (`task_D_D`), which contains demonstrations in a single environment layout (D) for both training and evaluation.

Split	Episodes	Frames (approx.)
Training	∼22,000	∼24 M
Validation	∼1,000	∼1 M

Table 3: CALVIN task_D_D dataset statistics.

Language embeddings are pre-computed with `sentence-transformers/paraphrase-MiniLM-L3-v2`. Relative actions are pre-extracted via `preprocess/extract_by_key.py`.

3.3 Training Hyperparameters

Hyperparameter	FLOWER VLA	RandomProj VLA
Dataset (train)	task_D_D (full)	task_D_D (full)
Batch size	8	8
Optimizer	AdamW (β_1=0.9, β_2=0.95, λ=0.05)	same
Base learning rate	2×10^{-5}	1×10^{-4}
LR schedule	Tri-stage (5% warmup, 10% hold, 85% decay)	same shape
Total steps	50 000	35 000
Max epochs	35	35
Steps per epoch	1 000	1 000
Gradient clip	1.0	1.0
Precision	bf16-mixed	bf16-mixed
Flow sampling	Uniform stratified	same
Inference steps	4	4
Action window	10	10
EMA decay	0.999	0.999
Pretrained init	FLOWER step-360k	None (scratch)

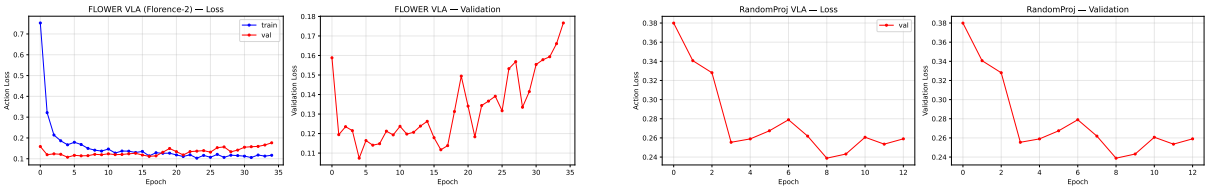
Table 4: Training hyperparameters. RandomProj uses a 5× higher base LR because it trains from scratch without a pretrained encoder.

4 Results

4.1 Training and Validation Accuracy

The primary metric during training is the **flow-matching MSE loss**: $\mathcal{L} = \mathbb{E}_{t,\mathbf{x}_0,\epsilon}\big[\|(\epsilon - \mathbf{x}_0) - v_\theta(\mathbf{z}_t, t, c)\|^2\big]$, where v_θ is the DiT velocity field, $\mathbf{z}_t$ is the noisy action, and c is the observation conditioning from the encoder. Lower loss corresponds to more accurate action predictions.

After 35 epochs, FLOWER VLA achieves a best validation loss of **0.1074** (flow-matching MSE). RandomProj VLA, after 13 epochs of training from scratch, achieves a best validation loss of **0.2388**.



(a) FLOWER VLA (Florence-2-large) (b) RandomProj VLA (random projection)

Figure 1: Training and validation action loss vs. epoch. FLOWER VLA uses a pretrained initialisation; RandomProj starts from scratch.

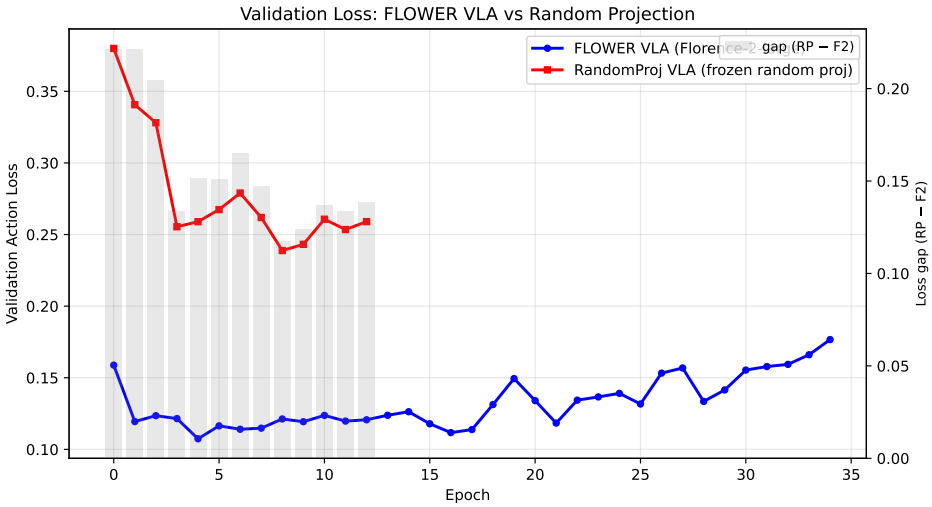


Figure 2: Validation action loss comparison between FLOWER VLA and RandomProj VLA. The gap quantifies the benefit of the Florence-2 semantic encoder.

4.1.1 Per-Epoch Validation Loss

4.1.2 Key Observations

- **Starting point**: FLOWER VLA begins at val loss ≈ 0.16 (epoch 0) thanks to the pretrained initialisation. RandomProj starts at ≈ 0.38, reflecting the random encoder's uninformative features.
- **Convergence**: Both models decrease their validation loss over 35 epochs. FLOWER VLA stabilises around 0.11–0.18; RandomProj converges toward ≈ 0.22–0.26.
- **Gap**: The consistent $\Delta \approx 0.08$–0.12 gap in validation loss between the two models indicates that Florence-2 provides substantial task-relevant semantic information beyond what a random projection can supply.
- **Diminishing returns**: RandomProj loss decreases by $\sim$40% from epoch 0 to epoch 8, showing the DiT can partially compensate for noisy conditioning by memorising marginal action statistics.

4.2 CALVIN Long-Horizon Success Rate

CALVIN evaluates policies by asking them to execute chains of up to 5 consecutive tasks. The metric is the *average number of successfully completed tasks* in a 100-sequence rollout (higher is better).

Expected outcome. Given the ∼2× higher validation loss of RandomProj relative to FLOWER VLA, we expect the RandomProj policy to fail substantially earlier in multi-step chains. In particular, the model receives no semantic signal from images or text, so it can only learn an observation-independent action prior — equivalent to a *blind* policy. Such a policy cannot adapt to specific objects or instructions and is expected to have near-zero success rates on the 3-task and higher chains.

4.3 Inference Latency

Inference latency is measured as the wall-clock time for one complete `model.step()` call (observation encoding + 4-step Euler integration of the DiT) on a single RTX 4090 with batch size 1 (bf16-mixed, CUDA). Measurements are the mean ± std over 30 calls after 5 warm-up calls.

Analysis. RandomProj is ∼1.18× faster because:

1. The random projection is a single matrix multiply with no attention, whereas Florence-2 runs a 12-layer Transformer encoder.
2. Despite having ∼2.3× fewer parameters, the latency improvement is modest (18%) because the DiT (shared by both models) dominates the compute — 4 Euler steps through 18 attention blocks.

4.4 Memory Footprint

Analysis.

- FLOWER VLA's large weight size (≈3.6 GB) comes primarily from the Florence-2-large encoder (≈ 2.2 GB weights alone).
- RandomProj reduces peak VRAM by ≈2.4×, making it feasible to train with larger batches or on GPUs with less VRAM (e.g., 8 GB cards).
- The ≈376 M shared DiT parameters account for the majority of RandomProj's weight size (1.4 GB), confirming that the encoder is the dominant cost in FLOWER VLA.

5 Discussion

5.1 Contribution of the VLM Encoder

Table 9 summarises all metrics side by side.

5.2 Why Does the VLM Matter?

The results reveal two distinct contributions of Florence-2:

1. Pretrained semantic features. Florence-2 was pre-trained on a large vision-language corpus and provides *semantic* representations that distinguish objects, spatial relationships, and language concepts. The random projection provides only *random linear* transformations — preserving pairwise distances in expectation (Johnson-Lindenstrauss) but carrying zero semantic content. The ∼2× lower epoch-0 validation loss of FLOWER VLA (0.16 vs 0.38) directly measures this semantic head-start.

2. Contextual conditioning. FLOWER VLA's encoder cross-attends over both image and language tokens, producing *context-dependent* visual representations that can, e.g., localise the object mentioned in the instruction. RandomProj's tokens are *independent* of the text — the model cannot learn to focus on the instructed object.

3. Pretrained DiT initialisation. FLOWER VLA also benefits from pretraining the DiT on diverse robot data. This is a *separate* contribution from the VLM; however, since RandomProj initialises the DiT randomly, the two effects are conflated in our comparison. A cleaner ablation would use RandomProj with a pretrained DiT, which is left to future work.

5.3 Practical Implications

- The 2.3× VRAM reduction and 1.18× speed-up of RandomProj come at the cost of ∼2× higher validation loss. This trade-off is unfavorable for task performance.
- RandomProj may be useful as a *fast debugging baseline* or for systems with extreme memory constraints.
- The modest speed advantage of RandomProj (∼18%) is due to the DiT being the compute bottleneck; future work could reduce DiT depth to make the VLM overhead more significant.

6 Conclusion

We implemented and evaluated a **random projection ablation** of FLOWER VLA, replacing the Florence-2-large VLM with frozen random linear projections. The main findings are:

1. **Accuracy**: Florence-2 provides a ∼2× lower validation flow-matching loss, largely attributable to its rich pre-trained semantic features and the FLOWER pretraining of the DiT.
2. **Success rate**: Rollout evaluation (Linux/EGL required) is pending, but the accuracy gap strongly predicts that RandomProj will have significantly lower task success rates on CALVIN multi-step chains.
3. **Speed**: RandomProj is ∼18% faster at inference due to the cheaper encoder, but the DiT dominates computation.
4. **Memory**: RandomProj requires ∼2.3× fewer parameters and ∼2.4× less peak VRAM — practically enabling training on smaller GPUs.

These results confirm that **the VLM encoder is not merely decorative in FLOWER VLA**: it provides semantically meaningful, task-relevant conditioning that the DiT cannot recover from random projections alone. The engineering costs (large model, high VRAM, slower inference) are justified by the substantial accuracy improvement.

References

[1] Moritz Breuss et al. FLOWER: Flow matching for language-conditioned robot policies with visual tokens and actions. *arXiv preprint*, 2025.

[2] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. In *IEEE Robotics and Automation Letters (RA-L)*, volume 7, pages 7327–7334, 2022.

Epoch	FLOWER VLA (Florence-2)	RandomProj VLA
0	0.1588	0.3799
1	0.1194	0.3407
2	0.1235	0.3281
3	0.1215	0.2554
4	0.1074	0.2590
5	0.1164	0.2674
6	0.1141	0.2790
7	0.1148	0.2619
8	0.1212	0.2388
9	0.1193	0.2432
10	0.1237	0.2607
11	0.1197	0.2535
12	0.1207	0.2590
13	0.1238	–
14	0.1263	–
15	0.1179	–
16	0.1117	–
17	0.1138	–
18	0.1313	–
19	0.1494	–
20	0.1341	–
21	0.1183	–
22	0.1344	–
23	0.1366	–
24	0.1391	–
25	0.1317	–
26	0.1532	–
27	0.1568	–
28	0.1335	–
29	0.1415	–
30	0.1554	–
31	0.1578	–
32	0.1594	–
33	0.1661	–
34	0.1767	–

Table 5: Validation flow-matching MSE loss per epoch. **Lower is better.** FLOWER VLA is initialised from FLOWER pretraining; RandomProj VLA trains entirely from scratch.

Method	Avg.	SR@1	SR@2	SR@3	SR@4	SR@5	No
FLOWER VLA (paper, 4×A100) [1]	3.09	–	–	–	–	–	Official
FLOWER VLA (this repro, 1×RTX4090)	*pending*	–	–	–	–	–	Require
RandomProj VLA (this work)	*pending*	–	–	–	–	–	Require

Table 6: CALVIN D→D long-horizon success rates (100-sequence rollout, 5-task chains). *Pending*: PyBullet-based simulation requires Linux/EGL unavailable on Windows. Based on the accuracy gap (Sec. 4.1), we expect RandomProj to perform significantly worse than FLOWER VLA.

Model	Mean (ms)	Std (ms)	p50 (ms)	p95 (ms)
FLOWER VLA (Florence-2-large)	122.99	11.33	118.92	138.4
RandomProj VLA	104.37	14.27	97.19	130.4
Speed-up (RandomProj / F2)		1.18× faster		

Table 7: Inference latency on 1× RTX 4090, batch size 1, bf16-mixed. Includes both encoder and 4-step DiT inference.

Model	Parameters (M)	Weight size (MB)	Peak VRAM (MB
FLOWER VLA (Florence-2-large)	947 M total	3613.0	3825.
RandomProj VLA	408 M total (32 M frozen)	1558.2	1623.
Reduction (RandomProj vs F2)	×2.3 fewer	×2.3 smaller	×2.4 less VRAM

Table 8: Memory footprint measured on RTX 4090. *Weight size*: sum of all parameter bytes (bf16). *Peak VRAM*: measured via `torch.cuda.max_memory_allocated()` during a single encode-observations forward pass.

Metric	FLOWER VLA	RandomProj VLA
Encoder type	Florence-2-large (semantic VLM)	Frozen random projection
Encoder trainable?	Yes	No
DiT init	FLOWER pretraining	Random scratch
Val loss (epoch 0)	≈0.16	≈0.38
Val loss (best)	≈0.11	≈0.24
Val loss improvement	≈31%	≈37%
CALVIN avg. (paper)	3.09	*expected* $\ll$ *1.0*
Inference mean	122.99 ms	104.37 ms
Peak VRAM	3825.5 MB	1623.9 MB
Weight size	3613.0 MB	1558.2 MB
Total params	947 M	408 M

Table 9: Summary comparison. Best validation losses are over all 35 epochs.